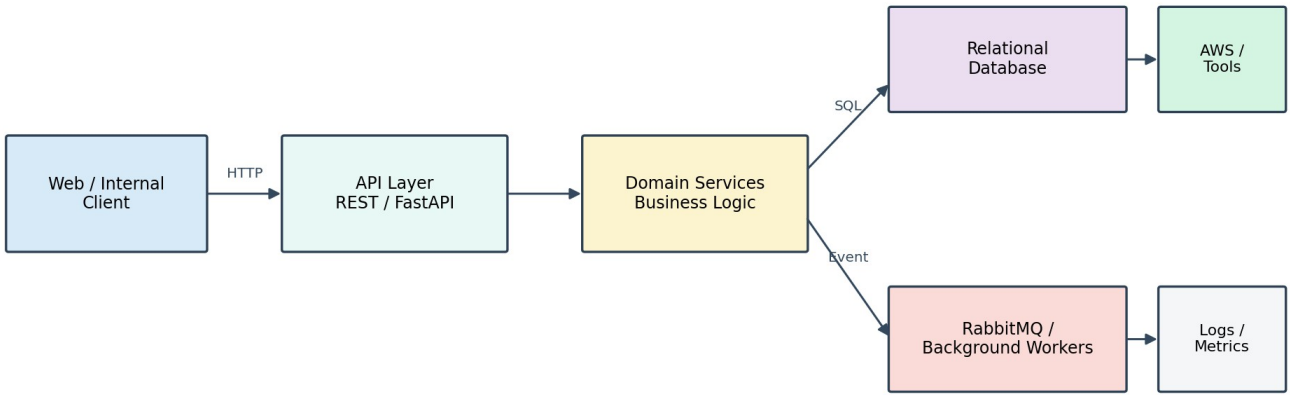
Backend API Design, Service Integration,

and Asynchronous Workflows

Engineering Case Study

Backend Service and Integration Context



Synchronous APIs handle bounded interactions; queues and workers isolate long-running or retryable work.

Author	Venkata Lakshmi Parimala Pasupuleti
Domain	Enterprise Backend Systems / Affordable Housing / AI Applications
Status	Experience-grounded engineering synthesis
Primary technologies	REST, FastAPI, SQL, relational databases, RabbitMQ, AWS integrations, Docker
Representative systems	Affordable Housing workflows, annual gross-rent processing, document migration, LeaseI
Scope boundary	Logical architecture; no invented proprietary service names or production topology
Document type	Technical case study - no interview questions
Version	1.0

Document Guide

Section group	Content
Context	Scope, business drivers, requirements, system boundaries, and accuracy labels.
API design	Resource modeling, request lifecycle, contracts, validation, authorization, and errors.
Integration	Synchronous calls, queues, data access, tools, external APIs, and object storage.
Reliability	Transactions, idempotency, retries, timeouts, failure handling, recovery, and observability.
Operations	Testing, deployment, scaling, incident response, trade-offs, and modernization.

NOTE	This document contains no interview questions. EXPERIENCE identifies work grounded in the engineering history discussed in this project. BUILT identifies directly implemented LeaseI capabilities. PATTERN identifies generally applicable design guidance. MODERNIZATION identifies proposed future-state improvements.

1. Executive Summary

My backend engineering experience is centered on enterprise workflows where APIs are only the entry point to a larger system of business rules, relational data, asynchronous work, documents, external services, and production operations. A successful backend service must preserve correctness across those boundaries rather than merely return an HTTP response.

Representative work includes database-intensive Affordable Housing modules, annual income-limit and gross-rent workflows, RabbitMQ-based processing, customer document migrations, AWS S3 integration, and the FastAPI and LangGraph application layer used by LeaseI. Across these systems, the recurring design concerns were clear contracts, controlled state transitions, bounded transactions, safe retries, evidence-based debugging, and operational visibility.

This case study describes how I structure backend APIs, choose between synchronous and asynchronous communication, isolate external integrations, model errors, protect database consistency, and modernize services without adding distributed complexity that the workload does not justify.

PRINCIPLE	An API is not a thin route handler. It is a controlled boundary that validates intent, enforces authorization, protects domain invariants, and produces observable outcomes.
EXPERIENCE	Enterprise backend development, SQL-heavy business logic, RabbitMQ processing, AWS document integration, production migrations, and FastAPI-based AI workflows.

2. Scope and Accuracy Boundaries

Capability	Status	What is supported
REST and backend business logic	EXPERIENCE	Built and maintained enterprise application workflows backed by relational data and domain rules.
RabbitMQ asynchronous processing	EXPERIENCE	Worked with queued annual gross-rent processing and developed knowledge of exchanges, queues, workers, acknowledgements, retries, and failure handling.
FastAPI application layer	BUILT - LeaseI	Endpoints accept document uploads, ticket submissions, and questions, then invoke the LangGraph workflow synchronously.
External tool/API integration	BUILT / EXPERIENCE	LeaseI includes a HUD income-limit client; enterprise work included AWS S3 document integration.
Formal API gateway and service mesh	MODERNIZATION	Proposed for larger multi-service environments; not claimed as existing production infrastructure.
Distributed tracing and centralized schema registry	MODERNIZATION	Recommended production maturity capabilities, not represented as fully implemented.
Microservices at scale	DESIGN / LEARNING	Containerized projects and reasoned about service boundaries; no claim of owning a large Kubernetes estate.

The document therefore uses logical components and broadly applicable names. It does not invent proprietary endpoints, service names, throughput figures, or infrastructure that were not provided.

3. Business Context and Backend Responsibilities

Affordable Housing software coordinates data and decisions across eligibility, certifications, income limits, rent calculations, HAP-related outputs, tax-credit requirements, property configuration, and customer documents. Backend services translate those workflows into controlled operations.

- Accept and validate user or system intent.
- Load the correct property, household, certification, and program context.
- Apply deterministic business rules and effective-date logic.
- Read and write relational data using explicit transaction boundaries.
- Invoke long-running work asynchronously when immediate completion is unnecessary.
- Integrate external systems through controlled adapters rather than leaking vendor behavior into domain logic.
- Return stable, explainable errors and preserve audit evidence.
- Expose enough operational signals for support and engineering teams to diagnose failures.

These responsibilities are more important than the specific web framework. FastAPI, PHP, or another application stack can implement the boundary, but the same domain and operational principles remain.

4. Functional and Non-Functional Requirements

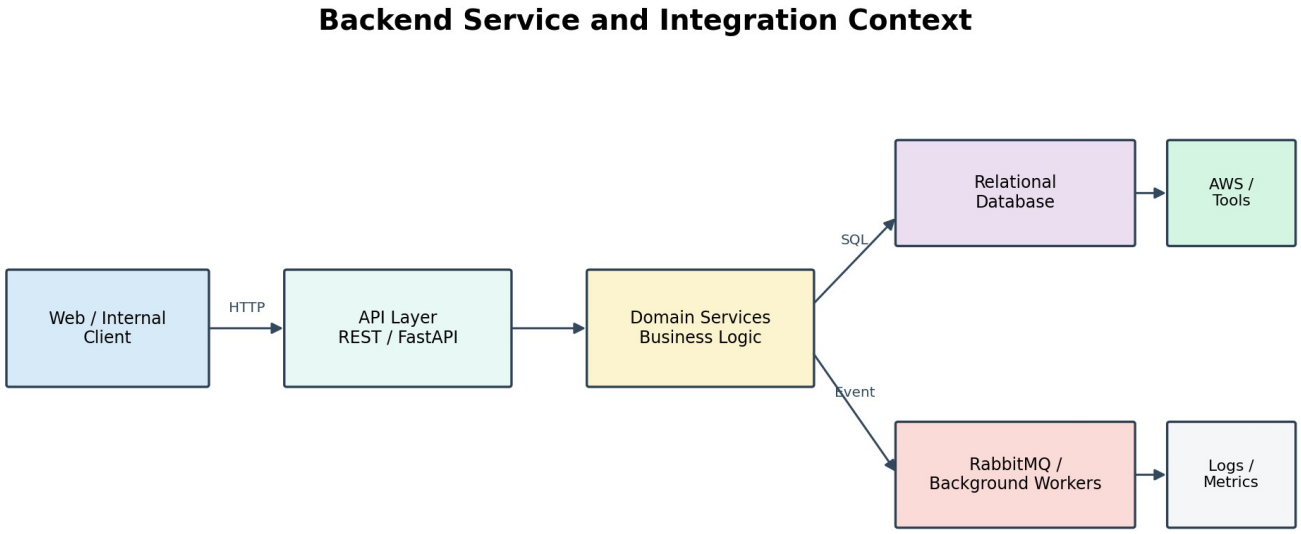
Functional requirements

- Create, retrieve, update, and investigate domain resources through stable contracts.
- Validate request structure and domain preconditions before changing state.
- Execute business logic within clear ownership boundaries.
- Publish background work when processing is long-running, high-volume, or independently retryable.
- Retrieve live authoritative values through approved read-only tools or APIs.
- Produce structured responses, errors, and escalation records.
- Preserve correlation between user request, database mutation, message, worker result, and audit record.

Non-functional requirements

- Correctness and consistency across database and asynchronous boundaries.
- Predictable latency for synchronous requests.
- Retry safety and duplicate tolerance for queued work.
- Backward-compatible contract evolution where consumers cannot move simultaneously.
- Least-privilege access to customer and resident data.
- Observability sufficient for production support.
- Maintainability for engineers who did not author the original workflow.
- Recoverability from partial dependency or worker failure.

5. Logical Architecture Overview



Synchronous APIs handle bounded interactions; queues and workers isolate long-running or retryable work.

Layer	Responsibility
Client boundary	Submits user or system intent and handles the public contract.
API layer	Authentication, validation, authorization, routing, response mapping, and correlation.
Domain services	Business rules, orchestration, invariants, and use-case ownership.
Data access	Queries, stored procedures, transactions, mappings, and persistence contracts.
Messaging	Decouples producers from long-running or retryable work.
Integration adapters	Encapsulate HUD, AWS, or other external service behavior.
Operations	Logs, metrics, traces, audit events, alerts, and support evidence.

The design intentionally separates transport concerns from domain rules. HTTP, RabbitMQ, and external APIs are mechanisms. The domain service owns the business decision and can be tested without requiring the transport layer.

6. API Boundary and Resource Modeling

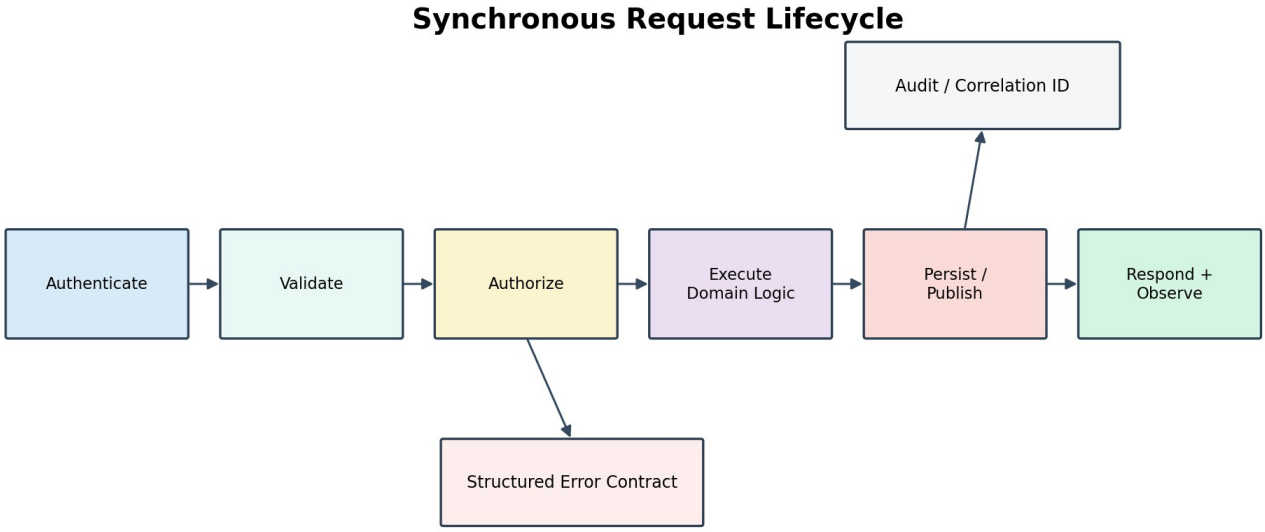
I prefer APIs modeled around business resources and actions rather than direct exposure of database tables. Resource-oriented contracts reduce coupling to schema changes and make business intent visible.

Contract type	Use	Example shape
Resource collection	Create or search domain resources	POST /investigations; GET /tickets
Resource instance	Read or update a known resource	GET /investigations/{id}
Explicit domain action	Represent a meaningful state transition	POST /imports/{id}/publish
Async submission	Accept work and return tracking identity	POST /gross-rent-jobs -> 202 + job_id
Read-only tool endpoint	Retrieve authoritative live data	GET /income-limits?...
Health / readiness	Expose runtime state without business data	GET /health; GET /ready

Not every verb should become a custom endpoint. An explicit action is useful when the operation has domain meaning, preconditions, audit requirements, or asynchronous execution that a generic update would hide.

PATTERN	The external contract should express business intent; internal persistence details should remain replaceable.

7. Synchronous Request Lifecycle



1. Authenticate the caller and attach an identity and correlation ID.

2. Parse and validate the request schema before entering domain logic.

3. Authorize access to the tenant, property, module, or operation.

4. Load required domain state and evaluate preconditions.

5. Execute the use case within an explicit transaction or orchestration boundary.

6. Persist state and, where required, publish follow-up work safely.

7. Map the result to a stable response and emit structured operational evidence.

A request should fail as early as possible for invalid input or insufficient access, but business conflicts should be evaluated with enough domain context to return an accurate explanation.

8. Validation and Domain Invariants

Validation occurs at several levels. Framework schema validation protects types and required fields, but it cannot enforce the full business contract.

Validation level	Examples	Failure handling
Transport / schema	Required fields, formats, ranges, enum values	Return structured 400 or 422 response.
Authorization	Tenant, property, module, and action access	Return 403 and record audit evidence.
Reference integrity	Property, household, program, ticket, or document exists	Return 404 or domain-specific validation error.
Business invariant	Effective date valid; state transition permitted; required configuration present	Return 409 or 422 with actionable details.
Cross-system precondition	External limit available; object exists; dependency healthy	Retry, degrade, or return dependency status based on operation.
Postcondition	Expected records, totals, or artifacts produced	Fail the operation or escalate before presenting success.

The most important rule is not to treat syntactically valid input as semantically valid. In Affordable Housing systems, correct values can still be invalid when applied to the wrong program, property, year, or certification state.

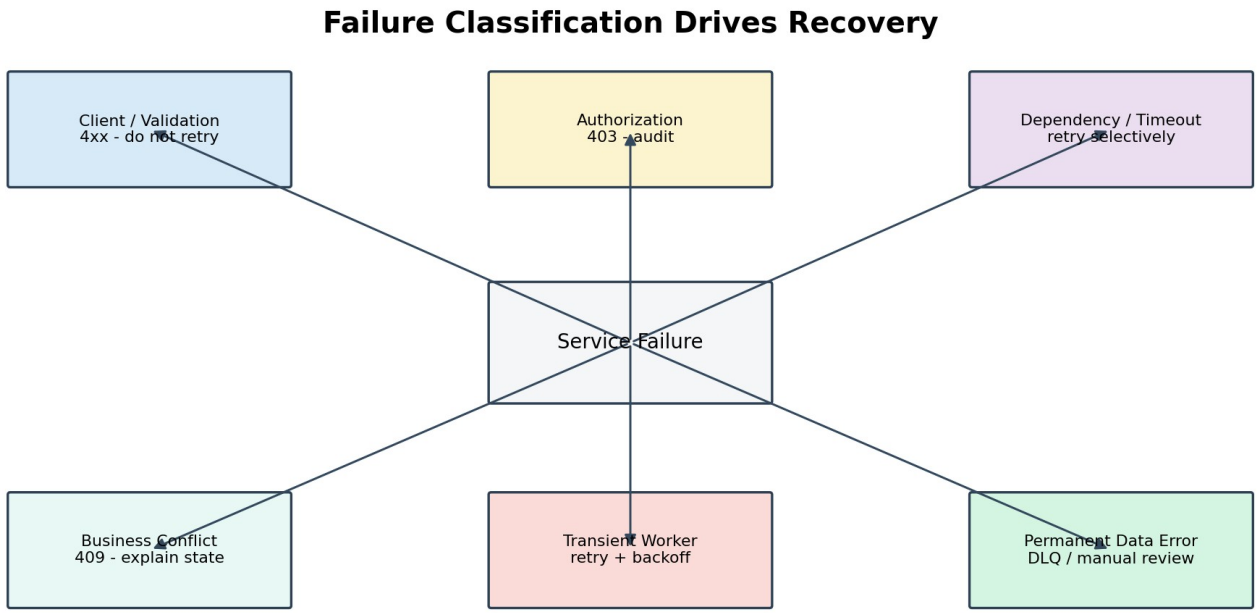
9. Authentication, Authorization, and Tenant Isolation

Customer, property, and resident information requires strict access boundaries. Authentication establishes identity; authorization evaluates whether that identity may perform a specific operation on a specific resource.

- Enforce tenant or customer scope in the service and data-access layers, not only in the UI.
- Use least-privilege service identities for databases, queues, object storage, and external APIs.
- Distinguish read-only diagnostic operations from state-changing operations.
- Audit sensitive reads and all production mutations.
- Avoid returning resident PII in logs, error payloads, or AI prompts unless explicitly required and protected.
- Validate object-storage paths and metadata to prevent cross-property access.
- Keep AI tool access allowlisted and read-only for diagnostic workflows.

LEASEI SAFETY	LeaseI is designed as a read-only diagnostic system. The AI layer does not directly change production settings or make final eligibility decisions.

10. Error Contracts and Failure Classification



Failure class	Typical response	Retry behavior
Invalid client request	400 / 422 with field and rule details	Do not retry unchanged request.
Authentication / authorization	401 / 403	Do not retry without corrected identity or access.
Missing resource	404	Retry only if eventual creation is expected.
Business conflict	409 with current state and permitted next action	Caller must reconcile state.
Transient dependency	502 / 503 / 504 or queued retry	Use bounded retries with backoff and jitter.
Permanent dependency data error	Structured failure or DLQ	Manual correction or source remediation.
Unexpected internal failure	500 with correlation ID	Do not expose internals; preserve evidence for investigation.

Stable error codes are more useful than prose alone. They allow UI, support, monitoring, and automated clients to respond consistently while the human-readable message explains the next step.

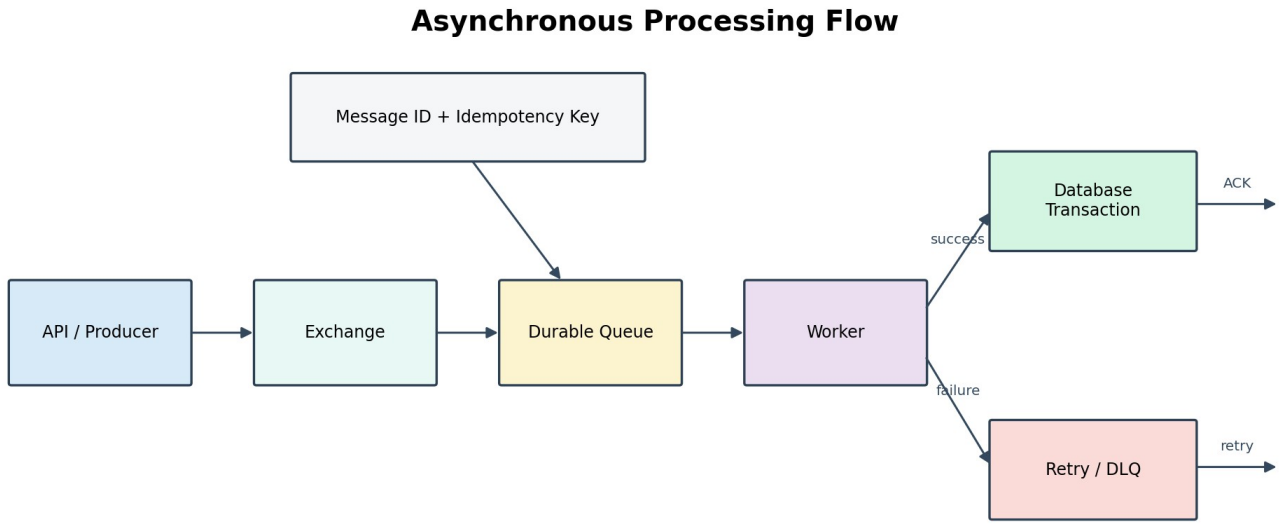
11. Synchronous Versus Asynchronous Processing

The central integration decision is whether the caller must wait for completion. I use synchronous execution when work is bounded, predictable, and necessary to complete the interaction. I use asynchronous execution when work is long-running, high-volume, retryable, or better isolated from user-facing latency.

Decision factor	Synchronous API	Asynchronous queue / job
User needs immediate result	Strong fit	Weak fit unless polling or notification is acceptable
Execution duration	Short and bounded	Long or variable
Failure recovery	Return error to caller	Retry independently and preserve job state
Load spikes	Caller absorbs latency	Queue smooths producer and worker rates
Atomicity	Single request transaction may be sufficient	Requires explicit job, message, and idempotency design
Operational complexity	Lower	Higher - queue health, workers, retries, DLQ, monitoring

RabbitMQ was appropriate for annual gross-rent processing because thousands of tenant updates should not block a user request or fail as one monolithic operation. The queue separated submission from execution and allowed workers to process individual units of work.

12. Asynchronous Workflow with RabbitMQ



Acknowledgement follows committed work; retries must be safe and duplicate delivery must not corrupt state.

1. The producer validates the command and records enough state to make the work traceable.

2. The producer publishes a compact message containing identifiers, version information, and an idempotency key.

3. The exchange routes the message to a durable queue.

4. A worker retrieves the message and loads authoritative data from the database.

5. The worker executes business logic inside a controlled transaction.

6. After the transaction commits, the worker acknowledges the message.

7. Transient failures are retried with limits; permanent failures are isolated for investigation.

The message should identify work, not carry an uncontrolled snapshot of sensitive domain state. Loading current authoritative state in the worker reduces payload size and avoids stale or PII-heavy messages, though version checks may be required to prevent processing an unintended newer state.

13. Message Contract and Versioning

Field	Purpose
message_id	Unique identity for tracing and duplicate detection.
event_type / command_type	Identifies the expected worker behavior.
schema_version	Allows consumers to handle contract evolution safely.
tenant / property identifier	Establishes scope without embedding excessive data.
entity identifier	Points to the tenant, certification, document, or job being processed.
effective context	Captures year, version, or date required for deterministic execution.
idempotency_key	Prevents duplicate side effects across redelivery.
correlation_id	Connects API request, message, worker, database action, and logs.
created_at	Supports age, delay, and stale-message analysis.

Contract evolution

- Prefer additive changes and default behavior over destructive renames.
- Keep producers and consumers compatible during rolling deployment.
- Validate schema version before executing business logic.
- Do not repurpose a field with a different meaning.
- Retain enough release history to explain messages created by an older producer.

14. Idempotency, Transactions, and the Dual-Write Problem

Message delivery is commonly at-least-once. A worker must therefore be safe when the same message is delivered more than once. Idempotency is a domain property, not merely a broker option.

Technique	Use
Processed-message table	Record message or idempotency key in the same transaction as the business update.
Natural uniqueness constraint	Prevent duplicate domain records at the database level.
State-transition guard	Apply work only when the entity remains in an expected state or version.
Upsert with business key	Make repeated writes converge on one intended result.
Deterministic artifact key	Avoid duplicate files or documents for the same operation.
Outbox pattern - modernization	Commit the domain update and pending event in one transaction, then publish asynchronously.

The hardest boundary is updating the database and publishing a message. Performing them independently can create a committed update without a message or a message without a committed update. A transactional outbox is a strong modernization path when this failure mode justifies the additional component.

PRINCIPLE	Acknowledge queued work only after the durable business outcome has committed.

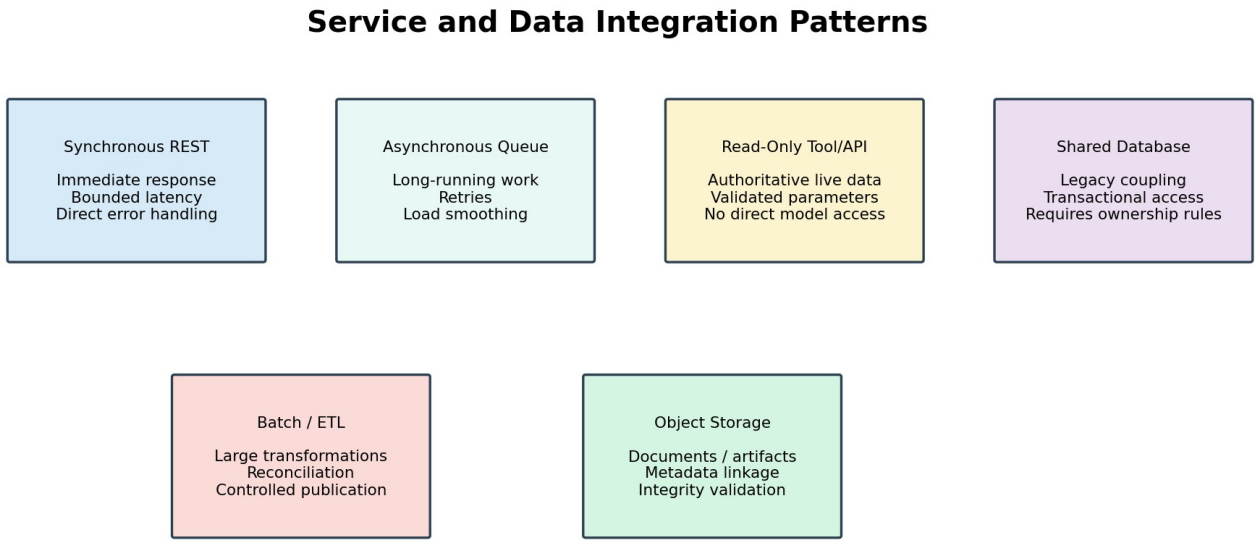
15. Retry, Backoff, Timeouts, and Dead-Letter Handling

Retries help only when the failure is transient. Retrying invalid business data wastes capacity and can hide the true problem.

Mechanism	Guidance
Timeout	Bound every network and database operation; unlimited waits consume workers and obscure failure.
Retry count	Use a small, explicit limit based on dependency behavior and business urgency.
Backoff	Increase delay between attempts and add jitter to avoid synchronized retry storms.
Retry classification	Retry timeouts and transient availability failures; do not retry validation or authorization failures.
Dead-letter queue	Isolate messages that exceed retry policy or fail permanently.
Manual replay	Require corrected cause, audit identity, and idempotent execution before replay.
Poison-message alert	Alert on repeated failure patterns rather than letting a queue silently accumulate.

A retry policy should be owned by the workflow, not copied blindly across every dependency. HUD lookup, S3 access, database deadlock, and permanent configuration errors have different recovery behavior.

16. Service-to-Service Communication Patterns



Pattern	When to use	Primary risk
Synchronous REST	Immediate bounded result from an independently owned service	Latency and cascading failure
Asynchronous messaging	Decoupled, retryable, high-volume, or long-running work	Eventual consistency and operational complexity
Read-only tool / API	Authoritative changing values such as HUD limits or settings	Dependency availability and parameter correctness
Shared database	Legacy modules with transactional coupling	Hidden ownership and schema coupling
Batch / ETL	Large controlled transformations and annual publication	Partial publication and reconciliation gaps
Object storage	Documents and large immutable artifacts	Metadata drift, access control, and integrity

No communication style is universally superior. The correct choice depends on latency, consistency, ownership, retry needs, and operational capability. A senior design makes those trade-offs explicit.

17. Data Access and Relational Database Integration

Relational databases remain central because the workflows require transactional consistency, effective dating, joins across business context, and strong constraints. The API layer should not scatter raw SQL throughout route handlers.

- Keep data access behind repositories, query services, or domain-specific stored procedures.
- Use parameterized SQL and strict type handling.
- Keep transaction boundaries aligned with one business outcome.
- Avoid holding transactions open across remote API calls.
- Use indexes based on measured access paths and execution plans.
- Return only required columns and paginate unbounded collections.
- Preserve audit history for high-impact configuration or calculation changes.
- Make tenant and property scope part of every sensitive query.

Stored procedures can be appropriate for set-based transformations and database-intensive workflows, as demonstrated by the income-limits pipeline. Application services remain responsible for orchestration, authorization, and cross-system decisions.

18. External APIs, Tools, and Object Storage

External integrations should be isolated behind adapters that normalize authentication, request construction, response parsing, timeouts, retries, and error translation.

Integration	Design responsibilities
HUD income-limit API	Validate jurisdiction and household parameters, use bearer authentication, parse documented response shapes, preserve effective context, and fail safely when unavailable.
AWS S3 document storage	Generate controlled object keys, preserve metadata linkage, validate object existence and integrity, and enforce least-privilege access.
Property / configuration lookup	Use read-only access for diagnosis, return typed setting values, and preserve source and release context.
AI tool execution	Allowlist tool names, validate structured parameters, execute outside the model, and write results into workflow state.
Historical incident search	Return source identity, resolution status, and similarity evidence rather than ungrounded summaries.

BUILT	LeaseI keeps model reasoning separate from external execution. Python nodes own database and HUD access; retrieved results are added to graph state before reasoning continues.

19. Observability and Operational Evidence

Backend observability must connect a user-visible symptom to the internal work that produced it. Metrics alone are insufficient without correlation and business context.

Signal	Examples
Request metrics	Rate, p50/p95/p99 latency, status codes, payload size, endpoint saturation.
Dependency metrics	Database duration, external API latency, timeout rate, S3 failures, queue publish failures.
Queue metrics	Depth, oldest-message age, publish rate, consume rate, retry count, DLQ count, worker concurrency.
Business metrics	Jobs submitted, entities processed, records updated, documents generated, reconciliation failures.
Structured logs	Correlation ID, tenant-safe identifiers, operation, state transition, error code, duration.
Traces - modernization	API span, database span, publish span, worker span, external tool span.
Audit evidence	Who performed the operation, what changed, source version, previous value, new value, and approval context.

For high-impact workflows, operational success should be defined by a business postcondition, not only an HTTP 200 or an empty queue. For example, an annual job is complete when expected records and downstream results reconcile.

20. Performance and Scalability

Performance work begins with measurement. I focus on identifying whether time is spent in request parsing, database queries, transformations, external dependencies, queue delay, or worker execution.

- Optimize relational access using execution plans, appropriate indexes, set-based operations, and reduced round trips.
- Paginate large reads and stream large files rather than buffering them unnecessarily.
- Cache only values with clear freshness and invalidation semantics.
- Move long-running work off the request path.
- Scale stateless API instances independently from workers.
- Scale worker concurrency against database and dependency capacity, not queue depth alone.
- Use backpressure and admission control to prevent a producer spike from overwhelming downstream systems.
- Measure end-to-end business completion time as well as component latency.

At enterprise application scale, a well-designed relational service and controlled queue may be more reliable than premature decomposition into many microservices. Scalability includes the team's ability to operate the architecture.

21. Testing Strategy

Test layer	Purpose
Unit tests	Validate domain rules, state transitions, mapping, and error classification.
API contract tests	Verify request schemas, response schemas, status codes, and backward compatibility.
Repository / database tests	Validate SQL, stored procedures, transaction behavior, constraints, and effective dates.
Integration tests	Exercise queue, worker, external adapter, and object-storage boundaries.
Idempotency tests	Deliver duplicate messages and confirm one intended business result.
Failure-injection tests	Simulate timeout, deadlock, unavailable dependency, partial file migration, and worker crash.
End-to-end tests	Verify user or support workflow from submission through durable result and audit evidence.
Security tests	Validate tenant isolation, authorization, sensitive logging, and tool allowlists.
AI evaluation - LeaseI	Measure retrieval, evidence support, escalation, and tenant-facing safety behavior.

Tests should validate both success and recovery. A workflow that works only when every dependency succeeds is not production-ready.

22. Deployment, Compatibility, and Rollback

Backend deployment must account for API consumers, database schema, producers, consumers, and external integrations moving at different times.

1. Classify release risk and identify contract, data, queue, or external dependency changes.

2. Use backward-compatible database migrations before application code depends on the new shape.

3. Deploy consumers that can tolerate both old and new message versions before switching producers.

4. Validate health, readiness, database connectivity, queue connectivity, and configuration.

5. Use phased rollout or feature flags for high-impact behavior where available.

6. Run technical and business post-deployment checks.

7. Rollback application behavior only when the data state remains compatible; otherwise use a prepared remediation path.

8. Retain evidence including release version, migration results, counts, and correlation IDs.

The Version 2 database migration experience reinforced that rollback decisions should be based on validated evidence. Timestamp differences, dumps, and migration records must be interpreted carefully before concluding that a migration is wrong.

23. Production Incident Response

Backend incidents often cross API, data, configuration, release, and asynchronous layers. Investigation should follow evidence rather than remain inside the module where the symptom appears.

- Capture the exact request, identifiers, timestamps, release version, and observed result.
- Trace the correlation ID through API logs, database changes, messages, workers, and external calls.
- Confirm whether the request reached the expected domain path.
- Compare live configuration and effective data with the assumptions used by the calculation.
- Inspect queue delay, retry history, worker failures, and duplicate delivery.
- Validate database state before and after the operation.
- Search historical incidents and release changes for matching symptoms.
- Contain impact before implementing a broad correction.
- Convert the confirmed root cause into documentation, monitoring, or a regression test.

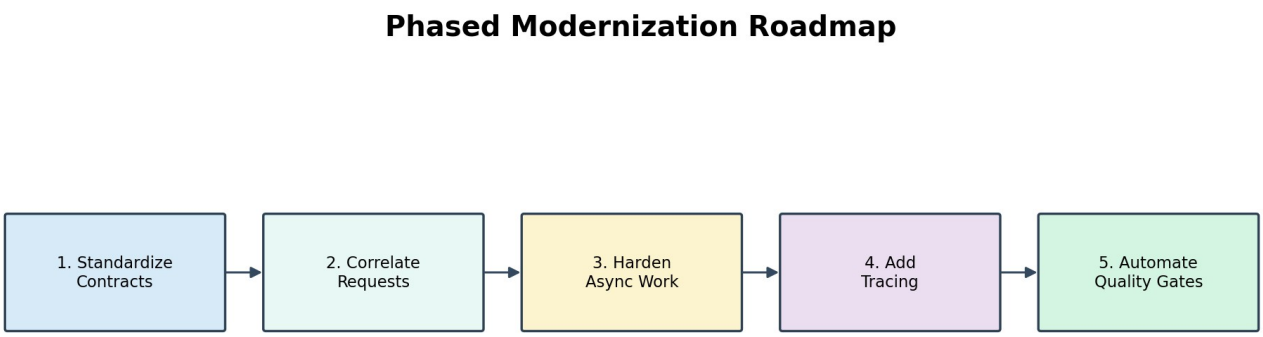
REAL INCIDENT	A nearly 24-hour eligibility investigation showed why backend diagnosis must include cross-team system settings. The Affordable Housing rules and data were correct; a setting introduced elsewhere changed the outcome.

24. Design Decisions and Trade-offs

Decision	Benefit	Trade-off
Relational database for core domain state	Transactions, constraints, joins, effective dating, mature tooling	Schema coupling and migration discipline required
Stored procedures for large set-based transformations	Efficient database-local processing	Logic distribution and specialized maintenance
REST for bounded service interactions	Simple request-response semantics and broad tooling	Temporal coupling and cascading latency
RabbitMQ for long-running work	Load smoothing, independent retries, worker scaling	Eventual consistency and operational overhead
Compact messages with identifiers	Lower PII exposure and smaller contracts	Worker must load authoritative data and handle version drift
FastAPI for LeaseI application layer	Typed schemas, rapid Python integration, async-friendly framework	Production authentication and background execution still required
Read-only AI tools	Safer diagnostic automation and auditable evidence	Human or deterministic systems must perform final action
Incremental modernization	Lower migration risk and preserves proven logic	Temporary coexistence and slower architectural convergence

The strongest architecture is not the one with the most components. It is the one whose failure behavior, ownership, and operational cost are appropriate for the business problem.

25. Current Limitations and Modernization Priorities



Improve contracts, evidence, and recovery before adding distributed infrastructure for its own sake.

Priority	Improvement	Outcome
1	Standardize API and message error codes, schemas, and versioning rules	More predictable clients and safer contract evolution
2	Propagate correlation and causation IDs across HTTP, SQL audit, queue, worker, and tool execution	Faster end-to-end incident diagnosis
3	Introduce durable job records, bounded retries, DLQ workflow, and replay controls	Safer asynchronous operations
4	Adopt transactional outbox where database-and-message dual write is material	Reduced lost or duplicate follow-up work
5	Add distributed tracing and business-level service objectives	Evidence across service boundaries
6	Run contract, migration, idempotency, and AI evaluation gates in CI/CD	Lower regression risk
7	Move LeaseI investigations to background workers with status APIs when workload requires it	Non-blocking execution and independent scaling

26. Reusable Engineering Standards

API standard

- Every endpoint has a documented resource or domain action, typed request, typed response, stable error codes, and authorization rule.
- Every state-changing operation has an idempotency or conflict strategy.
- Every request has a correlation ID and produces structured operational evidence.
- Every unbounded collection is paginated or streamed.
- Every dependency call has a timeout and explicit recovery policy.

Asynchronous-work standard

- Every message has an identity, schema version, correlation ID, and business scope.
- Workers are idempotent and acknowledge only after committed work.
- Retries are bounded and classified; permanent failures are visible and recoverable.
- Queue depth, age, throughput, retry count, and DLQ count are monitored.
- Manual replay requires corrected cause, authorization, audit evidence, and duplicate safety.

Integration standard

- External systems are accessed through adapters with validated parameters and normalized errors.
- Credentials are stored outside code and scoped to the minimum required access.
- Changing authoritative data is fetched live or cached with explicit freshness rules.
- AI models never receive unrestricted direct access to production systems.
- Object storage and database metadata are reconciled so artifacts do not become orphaned.

27. Capability Matrix

Area	Experience / current state	Next maturity step
Enterprise APIs	Backend workflows and business logic backed by relational data	Standardized contracts, versioning, and correlation across services
FastAPI	LeaseI synchronous API layer built	Authentication, background jobs, status polling, rate limiting
RabbitMQ	Annual gross-rent asynchronous processing experience	Formal idempotency store, DLQ operations, outbox, queue SLOs
Database	Strong SQL, stored procedures, migrations, validation, production diagnosis	Repository consistency, automated migration gates, expanded observability
AWS integration	Affordable Housing document migration integration and validation	Automated integrity reconciliation and richer operational dashboards
Tool calling	HUD and database access owned by Python nodes, not direct model access	Formal tool schema, policy enforcement, timeout and retry framework
Observability	Evidence-driven production debugging	End-to-end tracing and business SLOs
Security	Read-only LeaseI boundary and least-privilege principles	Production RBAC, tenant filters, PII policy enforcement, audit review

28. Lessons Learned

- Backend correctness depends on the full workflow, not only the route handler.
- Synchronous and asynchronous communication solve different problems; either can be harmful when chosen by habit.
- At-least-once delivery makes idempotency a required domain design.
- Business postconditions are stronger than technical success signals.
- Configuration and release history are part of runtime behavior and must be observable.
- External APIs should be treated as unreliable dependencies even when authoritative.
- Production support improves when requests, messages, data changes, and releases share correlation evidence.
- AI tools are safer when execution is controlled by deterministic orchestration and remains read-only for high-impact workflows.
- Modernization should reduce operational risk before it increases architectural sophistication.

SUMMARY	My backend engineering approach combines clear contracts, deterministic domain logic, relational consistency, selective asynchronous processing, controlled integration, and production evidence. The goal is not merely to expose services, but to build workflows that remain correct and diagnosable under failure.